Language Models as Data Constructors

ECON 895 Spatial Techniques in Empirical Economics

Noel D. Johnson
George Mason University

October 5, 2026

Lab exam, 7:20–8:30

1. Open `ECON895.Rproj`.

2. Download `lab_exam.R` from Canvas and save it inside your `ECON895` folder. The paths in the file start with `data/` and work only from there.

3. Type your name at the top of the file.

4. The paper exam has the same five tasks and, on its last page, a list of functions you may need.

If your copy of the course repository is missing or broken, clone it again from `github.com/noeldjohnson/spatial-fall-2026-slides` before 7:20.

Rules and submission

- Closed book and closed model. No notes and no AI tools of any kind. R's help pages, such as `?st_buffer`, are allowed.
- Use the internet only to download the exam from Canvas and to submit it.
- I will call the time at 8:00 and at 8:20. At 8:20, restart R and run your file from top to bottom. It must run in a fresh session without errors.
- By 8:30, upload `lab_exam.R` to Canvas and hand in the paper exam. If Canvas fails, email the file to `njohnsoL@gmu.edu`.
- If you are stuck on syntax, write what you intended in a comment. Reasoning earns marks.

The break follows the exam, and the lecture starts at 8:45.

Plan for the rest of tonight

Bringing in large language models

Setting up a research project

Geocoding with a model

Exercise 1

After fall break

The exam is collected. From here the tools are on for the rest of the course.

Bringing in large language models

What changes tonight

- In Weeks 1–5 you wrote vector code, raster code, joins, and a georeference by hand. The lab exam covered that material.
- From tonight AI tools are permitted on all coursework, as the syllabus states.
- The work shifts from writing each line yourself to directing a model and verifying what it produces.

A model turns text into rows

- Tonight's use of a model is constructing data from text.
- Examples are place names turned into coordinates, archival prose into entities and dates, scanned tables into columns, and free-text occupations into categories.
- Each of these is a measurement, so the output has to be validated like any other measured variable.
- Tonight's demo is the simplest case. The model answers from memory with no source in front of it. In real work you hand it the source text, and the checks are the same.

Ferrara (2026), assigned for tonight, is the practitioner's guide for economic history. Korinek (2023), recommended, covers economics generally.

Setting up a research project

Claude Code runs in your terminal

Claude Code is an agentic tool, which means it reads the files in a project, writes and runs code, and checks the result.

Install once on macOS or Linux. Paste into Terminal.

```
curl -fsSL https://claude.ai/install.sh | bash
```

- It logs in with a Claude subscription or an API key. Windows has its own one-line installer on the same site.
- It asks before it edits a file or runs a command, and you can allow a command once or for the whole project.
- Every graded task in this course stays possible in PatriotAI, a chat window with no access to your files. There you paste code and output between the chat window and R yourself.

Git and GitHub

Git records snapshots of a project folder on your computer, called commits. GitHub keeps a copy of that history online, where coauthors and other machines can reach it. You have used `git pull` all term to get the course materials.

`git clone` *url*	copy a repository from GitHub to your computer
`git status`	list what has changed since the last commit
`git diff`	show those changes line by line
`git add` *file*	mark a change to go into the next commit
`git commit -m "`*message*`"`	record a snapshot with a short description
`git push`	send your commits to GitHub
`git pull`	bring down commits made elsewhere

Commit before each session with a model. If the session goes wrong, `git diff` shows what changed and `git restore` returns the files to the last commit.

What stays out of a repository

- A public repository can be read by anyone. A private one is visible only to the people you invite, and GitHub offers private repositories on free accounts. A paper built on restricted or licensed data belongs in a private one.

- A file called `.gitignore` lists files git never tracks. Use it for large files, since GitHub warns above 50 MB and refuses files over 100 MB, for outputs your scripts regenerate, and for data or PDFs you do not have the right to redistribute.

- Passwords and API keys never go in a repository. Keep them in a file such as `.Renviron`, list that file in `.gitignore`, and read the key in R with `Sys.getenv()`.

- Deleting a file in a later commit leaves it in the history. A key that reaches GitHub has to be revoked and replaced.

A template you can clone

I start my own papers from a public template. These commands copy it into a new folder called `my-paper`, replace its git history with a fresh one of your own, and start Claude Code inside it.

Paste into Terminal

```
git clone https://github.com/noeldjohnson/research-workflow-template_shared.git my-paper
cd my-paper
rm -rf .git && git init
claude
```

- It holds a folder layout, a `CLAUDE.md` to fill in, nineteen rules, seven agents, sixteen skills, the hooks that enforce some of the rules, and templates for logs and notes.

- The next slides open each piece in turn. The handout `handouts/research_project_setup.pdf` carries the same tour on one page.

Each folder holds one stage of a paper

```
Data/raw/           as downloaded, never edited
Data/processed/     built by scripts from Data/raw
scripts/            R scripts, numbered and run in order
output/             intermediate results saved as .rds
Tables/ Figures/    written by scripts, never edited by hand
Paper/              main.tex reads Tables/ and Figures/
notes/              reading notes and research notes
explorations/       exploratory work, held to a lower standard
quality_reports/    plans, session logs, review reports
```

Work flows one way, from raw data to the paper. Everything downstream of `Data/raw/` can be deleted and rebuilt by running the scripts.

The layout is a replication package from the first day

- A replication package holds the raw data with its provenance, the code that runs in order, and the outputs the paper reports. This layout already has all three.
- Add two files early. A `README` names each data source, its version, and the date you downloaded it, the way Week 5 recorded GAEZ. A master script runs every other script in order.
- Economics journals now have a data editor run the package before they publish. A project organized this way needs little extra work to pass that check.

CLAUDE.md is the project's standing brief

Claude Code reads `CLAUDE.md` at the start of every session. The template's version asks for this.

```
## Research Summary
**Question:** [What is the main research question?]
**Hypotheses:**
- H1: [First hypothesis]
**Data strategy:** [What data will you use? Where will it come from?]
**Geography/Context:** [Where and when does your study take place?]

## Current Project State
| Component | Status | Notes |
```

- Keep the project-state table current, and each session starts where the last one ended. `/init` drafts a first version from the folder.

Rules hold conventions that travel between projects

A rule is a short file in `.claude/rules/`. Claude Code loads it like part of `CLAUDE.md`. Here is the top of the template's R conventions.

```
---
paths:
  - "scripts/**/*.R"
---
# R Code Standards
- set.seed() called ONCE at top (YYYYMMDD format)
- All packages loaded at top via library()
- All paths relative to repository root
```

- The `paths` line loads this rule only when the model works on an R script.

- The template's nineteen rules cover planning before acting, logging each session, replicating before extending, and when work is good enough to commit.

Agents are specialists you can run side by side

An agent is a file in `.claude/agents/` with its own instructions and its own list of allowed tools. Here is the top of the template's R reviewer.

```
---
name: r-reviewer
description: R code reviewer for research analysis scripts.
  Checks code quality, reproducibility, figure/table
  generation patterns, and theme compliance.
tools: Read, Grep, Glob
---
```

- The tools line makes it read-only. It can report and it cannot edit.

- Yesterday six reviewers read this deck at once. Three of them independently found that the September version blamed the model for an error our own validation code made. The geocoding section shows that error.

Skills are procedures you call by name

A skill is a folder in `.claude/skills/` holding a `SKILL.md`, and typing its name with a slash runs it.

```
---
name: data-analysis
description: End-to-end R data analysis workflow from
  exploration through regression to publication-ready
  tables and figures
disable-model-invocation: true
---
```

- `disable-model-invocation` means it runs only when you type `/data-analysis`. Without that line the model can also reach for a skill when its description fits the task.

- Others in the template include `/review-paper`, `/lit-review`, `/validate-bib`, and `/commit`.

Hooks run at fixed moments

A rule is text the model reads and should follow. A hook is a script that Claude Code runs itself at a set moment, whatever the model decides. The template wires them in `.claude/settings.json`.

before an edit	protect-files.sh blocks edits to the bibliography and the settings file
after an edit	verify-reminder.py asks for a recompile before the work counts as done
when the model stops	log-reminder.py asks for the session log to be brought up to date
before compression	pre-compact.py saves the current plan, and a second hook restores it after

The model can fail to follow a rule. It cannot skip a hook. Adding `Data/raw/` to `protect-files.sh` makes raw data read-only for the model.

Read a repository before you let it run

Hooks run code on your machine

Cloning a project with hooks and then starting Claude Code runs someone else's scripts with your permissions. Read `.claude/settings.json` and `.claude/hooks/` first. That applies to my template too.

- `settings.json` also lists commands the model may run without asking. Keep anything that deletes files or publishes them off that list until you trust the project.

- Commit before every session with a model, so `git diff` shows exactly what changed and `git restore` undoes it.

Habits that hold with or without a model

- Raw data is read-only, and every dataset carries its source, version, and download date.
- Every number in the paper comes from a script, and one command rebuilds them all.
- Paths are relative to the project folder, and random draws are seeded once.
- The project lives in git from the first day.
- A dated log records what you did and what you decided, so a coauthor or a referee can follow it.

Responsibility

You remain responsible for the question, the choice of sources, and the correctness of every result.

Geocoding with a model

The task

Twenty city names from the gazetteer you have used since Week 2. Seventeen are German and three Austrian, and the model gets the names alone, with no coordinates and no country.

The prompt, verbatim

```
Here are twenty European city names as they appear in a historical gazetteer. Give
me the latitude and longitude of each in decimal degrees, WGS84. Return a CSV with
columns name, lat, lon, confidence where confidence is high, medium or low. Do not
look anything up; answer from your own knowledge, which is the point of the
exercise.
```

The answer is frozen in `data/wk06_llm_geocodes.csv` with the model, the version, the date, and the prompt in the file header. Log the model the way you log a package version, because both determine the output. This one came from Claude in the session that built the deck.

in the chat window

Predict before you look

```
check <- llm %>% inner_join(truth, by = c("name" = "city_jjk"))

km_apart <- function(d) as.numeric(st_distance(
  st_as_sf(d, coords = c("lon", "lat"),              crs = 4326),
  st_as_sf(d, coords = c("longitude", "latitude"),  crs = 4326),
  by_element = TRUE)) / 1000
check$err_km <- km_apart(check)

data.frame(rows = nrow(check), within_25km = sum(check$err_km < 25),
           median_km = round(median(check$err_km), 1))
```

```
##   rows within_25km median_km
## 1   21          20       0.7
```

Each error is the great-circle distance in kilometers between the model's answer and the gazetteer's coordinates. The gazetteer is the gold standard tonight.

One answer is 546 kilometers off

```
check %>% arrange(desc(err_km)) %>%
  mutate(err_km = round(err_km, 1)) %>%
  select(name, countryname, lat, lon, confidence, err_km) %>%
  head(3) %>% as.data.frame() %>% print(row.names = FALSE)
```

```
##      name countryname   lat   lon confidence err_km
##     HALLE     Belgium 51.48 11.97     medium  546.2
##  MEISSEN     Germany 51.16 13.47       high    1.9
##     HALLE     Germany 51.48 11.97     medium    1.9
```

HALLE appears twice in the table, once in Belgium and once in Germany, with the same model answer on both rows. Decide whose error the 546 kilometers is before the next slide.

Twenty names went in and twenty-one rows came back

```
c(names_sent = nrow(llm), rows_returned = nrow(check))
```

```
##     names_sent rows_returned
##             20            21
```

The model was right and the check was wrong

The twenty names came from German and Austrian entries, so HALLE meant the German Halle, and the model placed it 1.9 kilometers away. The join matched the name to both Halles. The 546 kilometers measure the model's answer against the Belgian Halle, which was not among the twenty names.

The same duplicate-name trap appeared in Week 2 with ALBA and HALLE and again in question two of tonight's exam. Here it appears a third time inside code written to check a model. Counting rows on both sides of the join exposed it.

The same check, live in Claude Code

A project built from the template holds the twenty answers and the gazetteer in `Data/raw/`, with `CLAUDE.md` filled in. Claude Code gets one request.

The request

```
Write scripts/01_validate.R. It should measure how far each geocode in
Data/raw/llm_geocodes.csv is from the gazetteer coordinates in
Data/raw/europe_cities_1801.csv, then report the median and the worst error
overall and by confidence label. Run it and show me the output.
```

- Read the script before the numbers, and count the rows on both sides of its join.
- Then add a three-line rule about joins to `.claude/rules/`, start a fresh session, and ask again.
- The project ships as `examples/geocode_project/`, with the rule beside it in `examples/joins_rule.md`.

in the terminal

Exercise 1

Due date, files, and rules

Due by email to me at 7:20 PM on Monday, October 19, after fall break. It should take about an evening.

Each half is worth ten points

The build half asks you to make something. The audit half asks you to find what is wrong with something a machine made.

- Run `git pull` tonight. The handout is `handouts/exercise1.pdf` and the audit script is `code/exercise1_audit.R`.
- AI is permitted and expected on both halves. Every submission carries a short note saying what you asked for and what you checked.
- The build script must run in a fresh session. I will run it.

The build half

- Week 2 instrumented press adoption with distance from Mainz, where printing began. Wittenberg is where Luther published the Ninety-five Theses in 1517, and Becker and Woessmann (2009) use distance from it to instrument for Protestantism.

- Build distance from Wittenberg for every city, estimate the first stage within three distance cutoffs of your choosing, and write one paragraph on whether relevance holds.

- Answer the question the numbers support. If the instrument fails, say so and show why. That is a complete answer and earns full marks.

The audit half

The audit script reads like code a model writes from a one-sentence description of a Week 2 task. Five errors of kinds you have met this term were planted in it.

It runs clean

It finishes in a fresh session without errors, and every number it prints looks ready for a draft. Some of the five errors change the estimates and some change what the script reports.

- For each error give the line, what it does, why nothing complained, the fix, and the check that would have caught it.

- The check is worth as much as the other four together.

After fall break

Week 7 and your paper

- No class on October 12. Exercise 1 is due by email at 7:20 PM on October 19.
- Week 7 is market access, least-cost paths, and Dijkstra on a real network. With the tools on, the exercise is specifying the algorithm and checking it.
- Week 5 asked what map your paper would digitize. Tonight adds a second question, which is what text you would turn into rows.
- Optional over the break, and ungraded. Clone the template for your paper and fill in the research summary in `CLAUDE.md`. The handout walks through it.

Tonight in one line

A model's output is a dataset, and the code that validates it needs checking too.

References

Becker, S. O. and Woessmann, L. (2009). Was Weber wrong? A human capital theory of Protestant economic history. *Quarterly Journal of Economics*, 124(2):531–596.

Ferrara, A. (2026). A practitioner's guide to using large language models and generative AI in economic history. NBER Working Paper 35374, National Bureau of Economic Research.

Korinek, A. (2023). Generative AI for economic research: Use cases and implications for economists. *Journal of Economic Literature*, 61(4):1281–1317.